C++ Programming Problem Set

Module: Function Overloading & Compile-Time Polymorphism

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Function Overloading & Name Mangling

Integrates: I/O, Operators, Reference Variables, Loops, Arrays & Strings

Instructions & Pedagogical Guidelines

This problem set tests your mastery of compile-time static polymorphism in C++. You will construct programs featuring overloaded function families that vary by parameter count, type, and ordering. You will analyze compiler resolution steps (exact match, type promotion, type conversion), avoid return-type-only compilation pitfalls, handle pass-by-reference overload resolution, and build multi-type data processing utilities.

Level 1: Overload Criteria, Type Variations & Compiler Resolution

Problem 1: Unified Geometry & Area Computation Engine

Compile-Time Polymorphism | Overload Signatures | Math

Implement an overloaded function family named `computeArea`:
1. `double computeArea(double radius)` — Circle area calculation.
2. `double computeArea(double length, double width)` — Rectangle area calculation.
3. `double computeArea(double base, double height, char shapeCode)` — Triangle area calculation when `shapeCode == 'T'`.
Prompt the user for inputs inside a menu loop in `main()` and invoke the corresponding overload.

Parameter Count Variations | Parameter Type Variations | Menu Control Loop

Call Site	Matched Overload & Calculated Area
`computeArea(7.0)`	`Circle Overload Called -> Area: 153.94 sq units`
`computeArea(10.0, 5.0, 'T')`	`Triangle Overload Called -> Area: 25.00 sq units`

Problem 2: Compiler Resolution Algorithm Inspector

Exact Match | Type Promotion | Type Conversion Ambiguity

Construct a program containing two overloaded functions: `void processVal(int x)` and `void processVal(double x)`. In `main()`, test calls with explicit types: `processVal(5)` (int), `processVal(5.5)` (double), `processVal('A')` (char -> int promotion), and `processVal(5.5f)` (float -> double promotion). Demonstrate how passing a type requiring ambiguous standard conversion (e.g., long vs float) causes a compile-time ambiguity error.

3-Step Resolution Algorithm | Integral Promotion | Ambiguous Call Diagnostics

Passed Arguments	Compiler Resolution Path
`processVal('A')`	`Promotes char 'A' (ASCII 65) to int -> Invokes processVal(int)`
`processVal(3.14f)`	`Promotes float 3.14f to double -> Invokes processVal(double)`

Problem 3: The Return-Type-Only Compilation Trap

Compiler Errors | Name Mangling Rules | Signature Design

Write a program demonstrating why C++ functions *cannot* be overloaded based solely on differing return types. Attempt to declare `int getValue(int x)` and `double getValue(int x)`. Document the exact compiler error message. Explain how Name Mangling encodes parameter types into internal symbol tables (e.g., `_Z8getValuei`), but omits return types, making distinguishing calls like `getValue(10)` impossible.

Level 2: Intermediate Overloading (Arrays, Strings & References)

Problem 4: Overloaded Multitype Array Display & Statistics Engine

Array Parameters | Function Overloading | Const Array

Implement an overloaded function family named `printAndSummarize`:
1. `void printAndSummarize(const int arr[], int size)` — Computes sum, min, and max for integer arrays.
2. `void printAndSummarize(const double arr[], int size)` — Computes average and range for floating-point arrays.
3. `void printAndSummarize(const std::string arr[], int size)` — Finds the longest and shortest strings in the array.
Test all three overloaded versions in `main()`.

```
// Overloaded Multitype Signatures: void printAndSummarize(const int arr[], int size); void printAndSummarize(const double arr[], int size); void printAndSummarize(const std::string arr[], int size);
```

Array Passed	Summary Analysis Output
`int arr[] = {12, 45, 3, 88}`	`Sum: 148 | Min: 3 | Max: 88`
`std::string arr[] = {"C++", "Overloading"}`	`Shortest: "C++" (3 chars) | Longest: "Overloading" (11 chars)`

Problem 5: Pass-by-Value vs Pass-by-Reference Overload Ambiguity

Reference Parameters | Overloaded References | Ambiguity

Declare two overloaded functions: `void modifyVal(int x)` (pass-by-value) and `void modifyVal(int &x)` (pass-by-reference). In `main()`, declare `int a = 10;` and attempt to invoke `modifyVal(a)`. Document why the compiler flags this as an ambiguous call. Show how to resolve the conflict by removing the pass-by-value overload or using distinct function names.

Pass-by-Value vs Reference | Ambiguous Call Signature | Const Reference Overloads

Call Site	Compiler Resolution Status
`modifyVal(a)`	`ERROR: Call to 'modifyVal' is ambiguous (matches value and reference)`

Problem 6: Sequence Order Overloading (Matrix & Vector Scaler)

Parameter Ordering | Sequence Overloading | Matrices

In C++, functions can be overloaded based on the order of parameter data types. Implement two functions:
1. `void scaleData(int scaleFactor, std::string &text)` — Repeats `text` by `scaleFactor` times.
2. `void scaleData(std::string &text, int scaleFactor)` — Truncates `text` to `scaleFactor` length and appends ellipses `"..."`.
Demonstrate how swapping argument positions during invocation seamlessly routes CPU control to different logic.

Parameter Order Difference | String Manipulation | In-Place Reference Editing

Call Sequence	Resulting Text Output
`scaleData(3, str) [int, string]`	`Repeated: "CodeCodeCode"`
`scaleData(str, 4) [string, int]`	`Truncated: "Code..."`

Level 3: Default Arguments Ambiguity, Structs & Capstone

Problem 7: Default Argument & Overloading Collision Resolution

Default Arguments | Function Overloading | Collision

Default arguments can collide disastrously with function overloading. Write a program declaring:
- `void display(int x)`
- `void display(int x, int y = 100)`

In `main()`, demonstrate that calling `display(50)` triggers a compiler ambiguity error. Show how default arguments act as implicit parameter variations, causing call-site collision, and implement the clean fix by consolidating both into a single default-argument function.

```
// Collision Trap Analysis: void display(int x); void display(int x, int y = 100); display(50); // ERROR: Compiler cannot decide whether to call 1-param or 2-param default!
```

Call Site	Compiler Evaluation
`display(50)`	`Ambiguous call: Matches display(int) AND display(int, int=100)`

Problem 8: Const Reference vs Non-Const Reference Overload Resolution

Const Ref Overloading | Temporary Objects | R-Values

Write two overloaded functions: `void printBuffer(std::string &s)` and `void printBuffer(const std::string &s)`. Test calls in `main()` passing both a mutable l-value variable (`std::string str = "Mutable"`) and a temporary r-value literal string (`"Temporary Read-Only"`). Explain how the compiler selects the non-const overload for modifiable l-values and the const overload for temporary r-values.

Const vs Non-Const Ref Overloading | L-Value vs R-Value Binding | Const Correctness

Passed Argument	Matched Overload Target
`printBuffer(mutableStr)`	`Invokes printBuffer(std::string&) [Non-const mutable]`
`printBuffer("Literal")`	`Invokes printBuffer(const std::string&) [Const read-only]`

Problem 9: Cumulative Capstone: Physics Trajectory & Force Simulator

Comprehensive Integration | Polymorphism | Structs & Arrays

Build a physics simulation engine using an overloaded `computeForce` function family:
1. `double computeForce(double mass, double acceleration)` — Newton's Second Law ($F = m \times a$).
2. `double computeForce(double mass, double velocity, double radius)` — Centripetal force ($F = m \times v^2 / r$).
3. `double computeForce(const double masses[], double distance)` — Gravitational force between two masses ($F = G \times m_1 \times m_2 / d^2$).

Process a batch array of simulation parameters in `main()` and display a summary report.

Polymorphic Physics Engine | Const Array Parameters | Multitype Overloads | Complete Module Capstone

Input Parameters	Computed Physics Force
`Mass = 10.0 kg, Accel = 9.8 m/s²`	`Linear Force: 98.00 N`
`Masses = [100.0, 200.0] kg, Distance = 5.0 m`	`Gravitational Force: 5.339e-08 N`